Spatio-Temporal Temperature Forecasting Using Machine Learning and Deep Learning: A Comparative Analysis of Models and Methodologies

Niranjan Nagabhushan(101069951)(2412263)

Submitted for the Degree of Master of Science in

Applied Data Science

Department of Computer Science
Royal Holloway University of London
Egham, Surrey TW20 0EX, UK
December 2, 2024

Declaration

This report has been prepared on the basis of my own work. Where other published and unpublished source materials have been used, these have been acknowledged.

Word Count: 12666

Student Name: Niranjan Nagabhushan

Date of Submission: 2nd December 2024

Signature: Niranjan Nagabhushan

Abstract

This dissertation presents a detailed exploration of predictive temperature modelling using advanced machine learning (ML) and deep learning (DL) techniques, with a focus on analysing a high-frequency geospatial weather dataset. The dataset, provided by the supervising professor, contains temperature data from multiple geographic locations and across time, presenting unique challenges involving spatial and temporal dependencies. The project aims to develop an accurate prediction framework for temperature by utilizing both traditional machine learning algorithms and modern deep learning architectures to harness these complex spatial and temporal relationships.

The project starts with a comprehensive data preprocessing phase, including spatial feature extraction, handling missing values, encoding categorical features, and normalizing numerical data. Key features such as latitude, longitude, hour of the day, distance to lakes and rivers, and altitude were selected for their potential impact on temperature prediction. The intricacies of working with spatial data, especially handling geospatial geometries and encoding distance-based features, are a critical aspect of the study.

Four machine learning models, namely Ridge Regression, Lasso, Support Vector Regression (SVR), and Light, were employed to predict temperature based on the engineered features. The models' performance was compared against more advanced deep learning architectures such as Convolutional Neural Networks (CNN), Long Short-Term Memory Networks (LSTM), and Graph Convolutional Networks (GCN). These deep learning models were utilized to capture nonlinear relationships in the data that traditional methods may not be able to fully exploit. Additionally, ensemble methods were used to integrate predictions from multiple models, resulting in improved performance.

The results of the project are assessed using common evaluation metrics, including Mean Squared Error (MSE), Mean Absolute Error (MAE), and the R^2 score. The deep learning models, particularly CNN and LSTM, significantly outperformed the traditional machine learning models in capturing complex spatial and temporal relationships. Ensemble learning techniques further enhanced the overall model performance, demonstrating that combining predictions from different models led to better accuracy.

This research contributes to the field of environmental data science by showing how a combination of ML and DL models can be leveraged for spatiotemporal forecasting. The findings have potential applications in meteorology, environmental monitoring, and resource management. Future work includes extending the feature set, exploring more advanced graph-based models, and developing real-time prediction systems. The research emphasizes the challenges of working with limited computational resources, managing hyperparameter tuning, and overcoming bottlenecks such as memory errors and lengthy model training times.

Contents

1. Introduction

1.1 Background and Motivation

Weather forecasting isn't just a scientific endeavour; it's something that affects all our lives. Think about it: whether we're deciding if we need an umbrella, if a farmer should plant crops, or if an energy company should anticipate higher demands, accurate weather predictions are vital. The impact of reliable temperature predictions stretches from something as personal as a daily commute to critical decisions that affect entire industries. In short, precise weather data is the cornerstone of many decisions we make, consciously or subconsciously.

Traditionally, forecasting weather has relied on models based on physical laws—physics equations that describe atmospheric processes. These models are powerful, but they're also limited by their complexity and the computational power required to solve them. Atmospheric models solve fluid dynamics equations that require an immense amount of computational resources and often need large-scale supercomputers. Moreover, even with sophisticated physical models, accurately predicting localized weather phenomena remains challenging due to the chaotic nature of weather systems. That's where machine learning comes in.

Machine learning (ML), with its ability to spot patterns that humans may miss, offers a new way to improve our ability to predict weather. This kind of data-driven forecasting taps into large sets of historical data to train models, giving us predictions that can sometimes be even more reliable than those from traditional physics-based methods. In other words, while physical models are constrained by the need to represent atmospheric processes mathematically, ML models simply need data—lots of it—and they learn from that data to make predictions.

The analysis utilized a high-frequency weather dataset provided by the supervising professor, containing temperature data over time across multiple geographic locations in the regions of Italy and Switzerland. The dataset's complexity; covering both time and space; made it challenging but also incredibly interesting. It was like piecing together a giant jigsaw puzzle, with each temperature reading acting as one piece. Capturing these relationships in both space (between different locations) and time (how temperature evolves) became the motivation for diving deeper into spatio-temporal modelling. These spatio-temporal relationships present challenges that go beyond standard time series problems or simple spatial modelling. They require sophisticated approaches capable of handling both temporal evolution and spatial dependencies.

Major driving factor behind this work is the growing impact of climate change, which is increasing the frequency and intensity of extreme weather events. More accurate weather forecasting can help mitigate the risks and prepare society for these events, making weather prediction not just a technological challenge but also a vital social and societal necessity.

Effective prediction models could help cities prepare better for extreme heat or cold spells, help farmers adapt their practices to changing weather patterns, and allow emergency services to allocate resources in anticipation of hazardous conditions. Machine learning's ability to use historical data and adapt quickly makes it a potentially powerful tool in this fight.

This dissertation aims to determine which types of machine learning models best capture the complexities of temperature variation in both spatial and temporal dimensions. The aim was to explore how different models, from the simple to the complex, handle this data. Can a basic linear model capture what's happening, or do we need the advanced architecture of a Spatio-temporal Graph Convolutional Network (ST-GCN)? Can Long Short-Term Memory (LSTM) models capture the temporal dependencies effectively, or are there other neural network architectures that can do a better job? This was the question that led this journey into comparing various models, each with specific strengths, such as ease of interpretation, and weaknesses, like limitations in capturing non-linear relationships. It was not just about reaching the right numbers but understanding the "why" behind the performance differences and digging into the complexities of these data-driven models.

1.2 Aims and Objectives

The goal of this research is to use machine learning to predict temperatures accurately. Achieving this goal required focusing on:

- Data Preparation: Cleaning, handling missing data, scaling features, and ensuring readiness for analysis.
- Exploration of Models: Applying a range of models, from Ridge and Lasso Regression to LSTMs, Temporal Convolutional Networks, and ST-GCNs.
- Training and Tuning: Adjusting model parameters using methods like Grid Search or Random Search to achieve optimal results.
- Model Evaluation and Analysis: Evaluating models based on metrics like MSE, MAE, and R^2, while also considering interpretability and computational efficiency.

1.3 Relevance to the Data Science Field and Personal Career Impact

Weather forecasting with machine learning blends environmental science, mathematics, and computer science. Working on spatio-temporal datasets sharpened my problem-solving skills and critical thinking, with valuable lessons about adaptability, handling spatial-temporal data, and making strategic trade-offs. This experience has demonstrated the significance of machine learning for real-world applications, extending beyond weather forecasting to transportation, finance, and health sectors.

2. Background and Literature Review

2.1 Introduction to Weather Forecasting Techniques

Weather forecasting has a rich history that spans centuries, evolving significantly from rudimentary observation-based methods to advanced, scientifically rigorous systems that we use today. Early weather forecasting largely relied on a combination of "people knowledge "and the careful observation of natural phenomena, such as the movement of clouds or wind directions. With technological advancements, empirical and physics-based models began to form the backbone of modern meteorology. Numerical Weather Prediction (NWP) models emerged, built upon the fundamental physical laws of conservation of mass, energy, and momentum that dictates atmospheric processes. However, the chaotic nature of the atmosphere, limits the accuracy of these physical models, especially in long-term forecasting, as minor inaccuracies can exponentially grow over time (Kalnay, 2003).
The development of supercomputers in the latter half of the 20th century led to a significant leap in weather forecasting accuracy, allowing for high-resolution simulations and more precise calculations. Despite this progress, NWP models continue to face challenges. These models require considerable computational resources and still struggle with capturing smaller-scale, localized weather events, like thunderstorms, due to the inherent need, to balance accuracy against computational feasibility. Going further, the complex interaction between different components of the earth system—such as the atmosphere, ocean, and land—often necessitates simplifications that reduce the precision of these forecasts, especially at the micro-scale. These limitations have made the way for data-driven methodologies as supplementary or alternative tools to traditional weather forecasting approaches.

2.2 Emergence of Data-Driven Approaches

With the emergence of big data and advancements in artificial intelligence, data-driven approaches like machine learning (ML) have become increasingly popular in meteorology. These approaches offer an alternative to traditional NWP models by utilizing large historical datasets to train algorithms that can uncover complex, non-linear relationships between meteorological variables (Bauer et al., 2015). Unlike physical models, which simulate weather processes based on atmospheric dynamics, ML models can autonomously learn patterns directly from data, potentially identifying relationships that are too subtle or complex for explicit modelling through physical equations (Schultz et al., 2021).

One key advantage of ML approaches is their flexibility. Traditional NWP models require deep expertise in atmospheric physics to build effective predictive systems, whereas ML techniques rely more on data quantity and quality to achieve high levels of accuracy. However, this data-driven advantage can also pose challenges, especially when working with insufficient or low-quality datasets, which can lead to subpar or biased models.

Additionally, a major criticism of ML models, particularly deep learning models, is their lack of interpretability. This "black box" nature can hinder their application in areas where understanding the reasoning behind a prediction is just as crucial as the prediction itself.

2.3 Literature on Machine Learning in Weather Forecasting

The application of machine learning in weather forecasting has expanded significantly in the past decade, offering several approaches that each have unique advantages and drawbacks, which we will be seeing further.

2.3.1 Linear Models: Ridge and Lasso Regression

Ridge and Lasso Regression were among the earliest ML models used for temperature prediction, valued for their simplicity and easy interpretation. Ridge Regression, with its regularization to reduce overfitting, is effective for short-term predictions (Yang et al., 2017). Lasso Regression, using L1 regularization, is especially useful when selecting relevant features in large datasets. However, their linear nature means they often fall short in capturing the complex, non-linear patterns inherent in weather data.

2.3.2 Support Vector Regression (SVR)

To address the limitations of linear models, Support Vector Regression (SVR) provides a powerful alternative. By using kernel functions, SVR can model both linear and non-linear relationships by transforming input data into higher-dimensional spaces (Yan et al., 2018). This approach makes it better at capturing daily temperature variations compared to linear models. However, SVR's computational intensity, particularly with large datasets, limits its scalability for high-resolution predictions.

2.3.3 Tree-Based Models: Random Forest and LightGBM

Tree-based models like Random Forest (RF) and LightGBM are well-suited for weather prediction due to their ability to handle non-linear relationships and missing data. Random Forest, by combining multiple decision trees, is effective in capturing interactions between meteorological variables (Wu et al., 2020). LightGBM, an optimized gradient boosting method, enhances efficiency by using a leaf-wise approach, which speeds up training and boosts accuracy, especially with high-dimensional datasets (Ke et al., 2017). In our experiments, LightGBM managed to balance speed and accuracy effectively for complex weather data.

2.3.4 Neural Networks for Spatio-Temporal Modelling

Deep learning models have emerged as strong tools for capturing the intricate spatial and temporal dependencies in weather data. Convolutional Neural Networks (CNNs) identify spatial patterns, while Long Short-Term Memory (LSTM) networks handle time-related sequences. The combination of CNN and LSTM has been successful in modelling weather data across regions by learning both spatial and temporal dynamics (Di et al., 2019).

A more recent approach is the Spatio-Temporal Graph Convolutional Network (ST-GCN). This model uses graph theory to treat weather stations as nodes connected based on proximity, effectively modelling spatial correlations along with temporal trends (Yu et al., 2021). This makes ST-GCN particularly effective for regional forecasting, where understanding relationships between neighbouring areas is crucial.

2.4 Gaps and Challenges in the Current Literature

Despite the progress made in using ML for weather forecasting, several challenges still exist. One of the most significant challenges is the need for large amounts of high-quality, labelled data. Many weather stations experience sensor malfunctions or outages, leading to gaps in the data. While imputation techniques can be used to fill these gaps, they may introduce biases that affect model accuracy.

Another challenge is the lack of interpretability, particularly with deep learning models. Unlike physical models, where every prediction step is based on well-understood physical principles, deep learning models are often seen as "black boxes" that provide accurate predictions without offering insight into the mechanisms underlying those predictions (Rudin, 2019). This lack of interpretability can be problematic in critical applications where stakeholders require a clear understanding of how predictions are made.

Furthermore, there is limited literature that provides a comprehensive comparison between different ML models for weather forecasting. While several studies have compared the performance of individual models, such as SVR versus Random Forest, there is a need for research that compares simpler models—like Ridge and Lasso Regression—with more advanced architectures, such as LSTMs and ST-GCNs, across different use cases and datasets (Zhang et al., 2020). Such comparisons are essential for providing practical guidance on model selection for specific weather prediction tasks.

2.5 Contribution of This Research

This research aims to address some of these gaps by providing a detailed comparative analysis of various machine learning models for temperature prediction. Specifically, we aim to:

1. Compare Simpler and Complex Models: By comparing Ridge Regression, Lasso Regression, SVR, Random Forest, LightGBM, and advanced deep learning models, we seek to highlight the advantages and limitations of each model in different contexts of weather forecasting.
2. Spatio-Temporal Forecasting: Evaluate models that capture both spatial and temporal dependencies, such as LSTMs and ST-GCNs, and compare them with more traditional time-series approaches to assess their effectiveness in spatio-temporal modelling.
3. Evaluate Model Trade-offs: Assess each model in terms of accuracy, interpretability, scalability, and computational efficiency. This will provide valuable insights into which models are most suitable for practical temperature forecasting under different resource constraints.

This comparative analysis is intended to help data scientists and meteorologists understand the trade-offs between various ML models, ultimately guiding them in selecting the most suitable model for their specific use case.

3. Dataset Description and Data Preparation

3.1 Dataset Overview

The dataset used in this research was provided by the supervising professor and contains high-frequency space-time temperature data. Unlike publicly available datasets, such as ERA5, this dataset offers higher temporal resolution, providing temperature readings at frequent intervals across different geographic locations. The data includes features such as temperature, latitude, longitude, altitude, distance to lakes, distance to rivers, hour of the day, day of the week, and other meteorological indicators. The inclusion of both spatial and temporal aspects allows for the exploration of advanced machine learning models that can capture dependencies across both dimensions.

To facilitate analysis, the dataset was divided into several distinct components, each focusing on specific geographical regions and temporal ranges. The data spans multiple years and contains detailed information about weather station characteristics, ensuring a diverse range of atmospheric conditions that can be effectively used to train the predictive models. The availability of precise location-based information also enabled the use of models that exploit spatial relationships, such as graph neural networks.

3.2 Data Collection and Feature Details

The dataset consists of hourly temperature readings collected from several weather stations spread across a particular region. Key features included are:

- Temperature (target variable): Temperature readings taken at frequent intervals from various stations. This serves as the target variable for prediction.
- Latitude and Longitude: Spatial coordinates representing the location of each weather station.
- Altitude: Height above sea level for each weather station, which influences local temperature and weather conditions.
- Distance to Lake and River: Geographical proximity to major water bodies, which can significantly affect temperature due to water's thermal properties.
- Temporal Features: This includes the hour of the day, day of the week, and month, capturing seasonal and diurnal variations that influence temperature changes.

The dataset offers a mix of numerical and categorical features, necessitating careful preprocessing to ensure compatibility with various machine learning algorithms. The focus was not only on predicting temperature but also on understanding the contributions of each feature to the model's output, aiding interpretability.

3.3 Data Cleaning and Handling Missing Values

A significant portion of the dataset required cleaning before it could be used for model training. This involved:

- Removal of Duplicates: Initial analysis showed multiple redundant records for some weather stations. These duplicates were identified using unique station IDs and timestamps and subsequently removed.
- Handling Missing Values: Missing values were handled in two ways:
    - Numeric Features: For numeric features, missing values were imputed using the median of each feature. Median imputation was chosen due to its robustness against outliers, which are common in environmental data.
    - Categorical Features: Missing values in categorical features, such as station name or type of water body, were replaced with a placeholder value ('missing'). This ensured that the categorical encoding processes handled all instances uniformly.

Handling missing data in temporal series proved particularly challenging, as temporal gaps can affect model learning. To address this, linear interpolation was applied to the temporal sequence, filling in the gaps based on adjacent values. This method is appropriate for hourly temperature data, as temperature variations are typically smooth over short timescales.

3.4 Feature Engineering and Transformation

Feature engineering was a critical step in this project, aimed at extracting meaningful information from the raw data to improve model performance. This included:

- One-Hot Encoding: Categorical features, such as the 'day of the week' and 'month', were one-hot encoded to enable machine learning models to process them effectively without implying an ordinal relationship.

- Scaling and Normalization: Numerical features were standardized using the StandardScaler from Scikit-Learn. Standardization transformed the features to have zero mean and unit variance, which is particularly beneficial for distance-based models such as Support Vector Regression (SVR).
- Distance Calculations: Features like 'distance to lake' and 'distance to river' were calculated using Haversine distance based on latitude and longitude, providing spatial context that improved the models' understanding of environmental influences.
- Temporal Feature Engineering: Additional temporal features were engineered to capture seasonal effects, such as creating features for daylight hours based on timestamp and latitude.

3.5 Data Splitting and Cross-Validation

The dataset was split into training and test sets using an 80/20 ratio to ensure that the models could generalize effectively. Given the temporal nature of the data, the split was done chronologically, using the earlier data for training and the more recent data for testing. This was intended to mimic a real-world scenario where a model trained on historical data is used to make future predictions.

A spatio-temporal cross-validation scheme was also implemented to better evaluate model performance. This cross-validation method accounts for both spatial and temporal dependencies by splitting data based on both locations and time. Traditional random cross-validation is not suitable here, as it can lead to data leakage, with training and testing sets having overlapped temporal patterns.

4. Machine Learning Methodologies

4.1 Overview of Machine Learning Techniques Used

The temperature prediction task presents significant challenges due to the inherent spatial and temporal dependencies in the dataset. To effectively address this, a wide array of machine learning methodologies was utilized. Each model offers a unique approach to handling the subtleties of this problem, from linear relationships to more sophisticated spatio-temporal modelling. The following machine learning methodologies were implemented:

- Linear Regression Models (Ridge and Lasso)
- Support Vector Regression (SVR)
- Ensemble Learning Models (LightGBM)
- Deep Learning Models (Convolutional Neural Networks (CNN), Convolutional LSTMs (ConvLSTM), Temporal Convolutional Networks (TCN), Graph Convolutional Networks (GCN))

Each technique was chosen based on its ability to model particular aspects of the dataset. Linear models were used to establish baseline performance and ensure interpretability, while more complex models such as GCNs and ConvLSTMs were employed to better capture the intricate spatio-temporal relationships within the data.

4.2 Linear Regression-Based Models: Ridge and Lasso

Linear models served as the starting point of our journey through various machine learning techniques. Ridge Regression and Lasso Regression were chosen due to their inherent simplicity and interpretability, as well as their potential to shed light on the relationships among the different features in the dataset.

- Ridge Regression is a linear regression model enhanced by L2 regularization. The regularization adds a penalty to the cost function, proportional to the square of the coefficient magnitudes. This penalization aids in preventing overfitting by shrinking coefficients, especially when dealing with multicollinearity—where predictor variables are highly correlated. In our case, geographical variables such as altitude, latitude, distance to rivers, and distance to lakes exhibited considerable interdependencies.

By adding the regularization term, Ridge Regression manages the risk associated with these correlations by distributing the influence among correlated predictors, rather than assigning excessive weight to any single predictor. This ensures more robust predictions and helps capture the collective influence of geographically related features.

- Theoretical Foundation:
  Mathematically, Ridge Regression solves the following optimization problem:

$$\text{Minimize } ||y - X\beta||^2 + \lambda||\beta||^2$$

where λ: lambda is the regularization parameter that controls the amount of shrinkage applied to the coefficients (β\beta).

- Interpretation of Coefficients:
  Despite reducing the overall effect of any single variable, Ridge Regression still provides an interpretive lens through which we can understand the influence of each feature. For instance, we noticed that altitude and latitude consistently showed significant contributions, highlighting the importance of these spatial elements in determining temperature.

- Challenges and Insights:
  One of the main challenges was optimizing the regularization parameter λ: lambda, which was done through cross-validation to ensure the model wasn't under- or over-regularized. While Ridge Regression was valuable for understanding feature interactions and their general effects on temperature, its limitation lies in its inability to handle more complex relationships, like non-linear dependencies.

- Lasso Regression

In contrast, Lasso Regression employs L1 regularization, which not only reduces coefficient magnitudes but also forces some coefficients to zero, effectively performing feature selection. This made Lasso particularly useful in reducing model complexity and focusing only on the most impactful features.

- Sparse Solutions and Feature Selection:

  By setting some coefficients to zero, Lasso creates a sparse solution, thereby automatically selecting a subset of predictors that are most significant for predicting temperature. This property is beneficial when we want a more interpretable and less overfitted model. For instance, features like day of the year, latitude, and altitude were retained, while less relevant features, like certain categorical descriptions of proximity to minor water bodies, were excluded.

- Implementation and Parameter Optimization:
  Lasso's λ\lambda parameter controls the degree of sparsity. Choosing an optimal value involved balancing between reducing model complexity and retaining essential features. The cross-validation approach enabled effective tuning of λ\lambda, ensuring that the model was not too simplified.

- Limitations:
  Lasso's inherent feature selection can sometimes discard features that, while collinear, might still be critical. This led to challenges when trying to balance simplicity with accuracy, especially in geographical settings where certain correlated features, like proximity to rivers and altitude, together influence temperature significantly.

4.3 Support Vector Regression (SVR)

Support Vector Regression (SVR) represents an important progression from linear models, incorporating non-linear interactions through the kernel trick. In temperature prediction, the relationships among the variables are often highly non-linear, making SVR well-suited for the task.

- RBF Kernel and non-linearity

SVR employs kernels to project data into a higher-dimensional space where a linear relationship can be found. The Radial Basis Function (RBF) kernel was particularly effective for our dataset as it can model non-linear relationships by mapping data to an infinite-dimensional space. This ability allowed SVR to better capture complex dependencies between temperature and other features, such as the interaction between latitude and elevation.

- Mathematical Formulation:
  SVR seeks to minimize:

$$\frac{1}{2}||w||^2 + C\sum_{i=1}^{n} L_\epsilon(y_i - f(x_i))$$

Here, Lϵ represents the epsilon-insensitive loss function, C is a regularization parameter, and f(xi)f(x_i) is the predicted value. The model uses support vectors to find a hyperplane that maximizes the margin while minimizing the prediction error.

- Dimensionality Reduction with PCA

One of SVR's primary challenges is the high computational cost, especially when applied to a high-dimensional dataset. To address this, Principal Component Analysis (PCA) was employed for dimensionality reduction. PCA reduces the number of features by extracting orthogonal components that account for the maximum variance in the data. By reducing the number of features while retaining approximately 85% of the variance, SVR became computationally feasible, especially during hyperparameter optimization.

- Hyperparameter Optimization and Insights

Key hyperparameters such as C and gamma were optimized using grid search with cross-validation. The C parameter controls the regularization, balancing the trade-off between maximizing the margin and minimizing classification errors, while gamma defines the influence of a single data point.

Despite the enhanced ability of SVR to handle non-linearity, the model faced scalability issues. As the dataset size increased, the training time grew exponentially, even after dimensionality reduction. This limits SVR's applicability for real-time applications or when working with highly detailed spatial data over extended periods.

- Results and Performance Metrics

The Mean Squared Error (MSE) and Mean Absolute Error (MAE) values were significantly lower for SVR compared to Ridge and Lasso, indicating better capture of variability in temperature. However, the R^2 score suggested that while SVR could model the data well, the trade-off between computational efficiency and accuracy still made it less practical compared to ensemble methods like LightGBM.

4.4 Ensemble Learning Models: LightGBM

Ensemble learning techniques take advantage of multiple weak learners to create a robust model. LightGBM, an efficient gradient boosting framework, was utilized in this study because of its speed and efficiency, especially in dealing with large datasets and complex interactions.

- Gradient Boosting Framework and Tree Growth

LightGBM is based on gradient boosting, which sequentially builds weak learners—typically decision trees—in such a way that each new tree aims to reduce the residual errors of the preceding ensemble. The unique aspect of LightGBM is its leaf-wise tree growth. Instead of growing trees level-wise, LightGBM grows trees by focusing on the leaf with the maximum loss reduction, allowing for deeper trees and a better fit for the training data.

    - Hyperparameter Tuning
        - Learning Rate (η\eta): A small learning rate (0.050.05) was used to gradually converge to a solution without overshooting the minimum error.
        - Number of Leaves: Many leaves helped LightGBM capture complex patterns in the data. However, to prevent overfitting, max depth was limited to 10.
        - Min Data in Leaf: This parameter was fine-tuned to control the minimum number of samples per leaf, balancing between model complexity and overfitting.
    - Feature Importance Analysis

One of LightGBM's major advantages is its feature importance capability, which provides insights into the contribution of each feature to the model's performance. Key features identified were altitude, latitude, distance to major water bodies, and time-related features such as hour and day of the year. This analysis highlighted the geographical and temporal influences on temperature, offering valuable interpretative insights.

- Results, Challenges, and Interpretations

LightGBM provided a substantial improvement in predictive power compared to linear and SVR models, yielding a significantly higher R^2 score and lower MSE and MAE. However, the model's performance was sensitive to hyperparameter tuning, and overfitting remained a risk if the model was too complex. Regularization techniques like L1/L2 regularization were crucial in overcoming this challenge.

4.5 Convolutional Neural Networks (CNN)

Convolutional Neural Networks (CNNs) were used to extract spatial features from the temperature dataset, treating it as a structured 2D grid. This was particularly useful for capturing temperature patterns across different geographic regions.

- CNN Architecture

The CNN architecture comprised multiple convolutional layers, each followed by a ReLU activation and max-pooling layer. The convolutional layers used 3x3 filters to detect spatial features, while max pooling helped reduce dimensionality, retaining only the most significant spatial features.

  - Activation Functions and Convolutional Layers

The ReLU (Rectified Linear Unit) activation function was used to introduce non-linearity into the model, enabling it to capture complex relationships that linear models could not. ReLU has the advantage of computational efficiency, as it reduces the likelihood of the vanishing gradient problem, which often plagues deep neural networks.

  - Fully Connected Layers

Following the convolutional layers, the fully connected layers were used to combine the spatial features extracted by the convolutional layers to make the final temperature prediction. This architecture allowed the model to understand how different geographical features combined to influence temperature.

- Strengths and Limitations

CNNs were effective in capturing local spatial dependencies, such as the influence of nearby water bodies or elevation changes on temperature. However, CNNs alone were not well-suited for capturing temporal dependencies, as they only process a static input at any given time. This limitation led to moderate R^2 scores for CNNs, as they failed to account for temporal patterns in the temperature data.

4.6 Convolutional LSTMs (ConvLSTM)

To overcome the limitations of traditional CNNs in handling temporal data, Convolutional LSTMs (ConvLSTMs) were implemented. ConvLSTMs extend the capabilities of CNNs by adding a temporal dimension, allowing them to learn both spatial and temporal dependencies simultaneously.

- Model Architecture
    - Input Representation:

      The input consisted of a sequence of spatial grids representing temperature readings at different time intervals. These grids were fed into ConvLSTM layers, which performed both convolutional operations to extract spatial features and recurrent operations to capture temporal dependencies.
    - ConvLSTM Layers:

      The ConvLSTM layers were designed to capture spatio-temporal relationships by combining the convolutional operations of CNNs with the gating mechanisms of LSTMs (Long Short-Term Memory). These gates control the flow of information, allowing the model to remember or forget information as needed, making ConvLSTMs particularly powerful in capturing temporal dependencies.
- Challenges and Implementation Considerations

Training ConvLSTMs was computationally demanding due to the need to model both spatial and temporal components. The model required substantial GPU resources and careful tuning of hyperparameters such as the number of filters, kernel size, and recurrent dropout rates to prevent overfitting while maintaining model performance.

- Performance and Spatio-Temporal Understanding

ConvLSTMs achieved the highest R^2 scores among all models, demonstrating their superior ability to capture complex, spatio-temporal relationships in temperature. However, due to their high computational requirements, ConvLSTMs may not be ideal for applications that demand real-time processing or when working with limited computational resources.

4.7 Temporal Convolutional Networks (TCNs)

Temporal Convolutional Networks (TCNs) are an innovative approach that combines the strengths of convolutional layers with a temporal perspective, providing a viable alternative to recurrent models like ConvLSTMs.

- Model Architecture

TCNs employ dilated causal convolutions, which enable the network to capture long-term temporal dependencies by expanding the receptive field without requiring a deep network. Dilated convolutions, unlike standard convolutions, skip over inputs at regular intervals, allowing the model to gather information from wider time spans without adding additional layers.

  - Residual Connections:

    One notable feature of TCNs is the use of residual connections. These connections add the input of a layer to its output, which helps in gradient flow during backpropagation and allows the network to train more efficiently. This aspect was particularly helpful in our model, as it allowed for deeper architectures that could better capture complex temporal relationships.

- Advantages Over Recurrent Networks

Unlike recurrent models, TCNs can be trained in parallel due to the use of convolutional layers, leading to faster training times. This property made TCNs more efficient compared to ConvLSTMs, especially for capturing seasonal and diurnal cycles in temperature.

- Results and Interpretations

TCNs demonstrated competitive performance, with a notable improvement in MSE and MAE compared to CNNs. The model was particularly effective in capturing long-term temporal patterns, which made it suitable for understanding how temperature evolves over extended periods, such as seasonal changes.

4.8 Graph Convolutional Networks (GCNs)

Graph Convolutional Networks (GCNs) were utilized to model spatial dependencies between different geographical locations represented as nodes. This approach allowed us to capture interactions between weather stations based on their proximity.

- Graph Representation

The geographic locations of weather stations were represented as nodes in a graph, with edges established using the k-nearest neighbors (KNN) approach. This representation helped in modeling how temperature at one location influenced its neighboring locations, effectively capturing spatial correlations.

  - Graph Convolutional Layers:
    GCNs employ convolutional operations on graphs, where each node aggregates information from its neighboring nodes to update its feature representation. This allowed the model to understand how spatial proximity and geographical features of neighboring weather stations impacted temperature.

- Strengths and Computational Challenges

GCNs were highly effective in regions where spatial relationships played a critical role, such as coastal and mountainous areas. However, the model's computational complexity increased with the number of nodes and edges, necessitating graph sampling techniques to maintain efficiency during training.

4.9 Comparative Analysis of Machine Learning Models

- Ridge and Lasso Regression provided an excellent baseline for understanding key features influencing temperature but were limited in their capacity to model complex relationships.
- Support Vector Regression (SVR) successfully captured non-linear dependencies but was computationally intensive, limiting scalability.
- LightGBM offered a balance between accuracy and interpretability, providing high accuracy with interpretable feature importance scores.
- CNNs captured local spatial dependencies effectively but lacked the ability to model temporal changes.
- ConvLSTMs offered the most comprehensive solution for spatio-temporal modeling, providing the highest accuracy but requiring significant computational resources.
- Temporal Convolutional Networks (TCNs) were efficient in capturing long-term temporal dependencies, offering a promising alternative to recurrent models.
- Graph Convolutional Networks (GCNs) excelled at understanding spatial relationships but required efficient graph construction to manage computational complexity.

5. Experimental Results and Analysis

The experimental results and analysis are crucial in assessing how effectively the models and methodologies predict temperature. This section covers the results from various models, including traditional regression models, Convolutional Neural Networks (CNN), Graph Convolutional Networks (GCN), and others. It highlights their performances, key insights, and the challenges encountered throughout the process. The analysis is supported by visualizations, metrics, and technical insights for a thorough evaluation.

5.1 Overview of Experimental Approach

The experimental approach was divided into three phases: preprocessing, model development, and evaluation. Each phase was designed to effectively handle both spatial and temporal components of the weather data, aiming to achieve accurate temperature predictions while accounting for spatial interactions. Preprocessing involved data cleaning, feature engineering, and normalization, while model development focused on selecting and training models that could best capture the spatio-temporal characteristics of the dataset.

5.2 Dataset Preprocessing

5.2.1 Raw Data Preparation

The dataset provided included high-frequency temperature data with spatial features like latitude, longitude, and elevation, along with temporal attributes such as day and hour. Preprocessing involved several key steps:

- Missing Value Imputation: Missing temperature data, especially from remote areas, were filled using kriging, a spatial interpolation technique that uses correlations to estimate values. Categorical features were filled with appropriate placeholders.
- Normalization and Standardization: Numerical features were standardized using StandardScaler to bring all values to a similar range, ensuring a balanced learning process. Temporal features like hour and day were normalized to highlight their periodic nature.
- Spatial Relationships: Spatial features were enhanced by calculating distances to nearby lakes and rivers using KDTree algorithms, allowing models to learn localized temperature variations. Spatial clustering also grouped similar geographic areas, adding meaningful context for predictions.

5.2.2 Challenges in Preprocessing

- Handling Missing Values: Filling in missing geographic data, especially with kriging, was computationally demanding but essential for maintaining spatial relationships.
- Data Cleaning: Outliers, particularly in elevation data, required manual inspection and filtering to avoid impacting model performance. Robust scaling was applied to mitigate the influence of these outliers.

5.3 Experimental Setup

To evaluate the performance of temperature prediction, various models were trained and compared. Each model was chosen based on its suitability for either spatial or temporal data, or both:

- Traditional Regression Models: Ridge and Lasso Regression were used as baselines, providing insight into the performance of simpler models that assume linear relationships.
- Deep Learning Models: More complex models, such as CNN, GCN, and Long Short-Term Memory Networks (LSTM), were used to capture non-linear patterns and spatio-temporal relationships in the dataset.
- Ensemble Learning: A weighted ensemble of multiple models was also used to improve robustness and accuracy by leveraging the strengths of each model.

Each model was evaluated using Mean Squared Error (MSE), Mean Absolute Error (MAE), and R^2 Score as metrics. These metrics provided insights into the models' accuracy and reliability. MSE penalized larger errors more severely, making it suitable when large deviations are undesirable, while MAE provided an interpretable average error magnitude.

5.4 Ridge and Lasso Regression

5.4.1 Model Motivation

Ridge and Lasso regression were employed as baselines. These models were chosen for their simplicity and ability to manage multicollinearity through regularization. Ridge regression uses L2 regularization to penalize large coefficients and reduce overfitting, while Lasso regression uses L1 regularization to perform feature selection by driving some coefficients to zero.

5.4.2 Results and Evaluation

- Performance Metrics:
    - Ridge Regression:
        - MSE: 0.925
        - MAE: 0.765
        - R^2 Score: 0.18
    - Lasso Regression:
        - MSE: 1.01
        - MAE: 0.801
        - R^2 Score: 0.12

The low R^2 scores indicated that both models struggled to capture the dataset's complexity. Ridge regression performed slightly better than Lasso, likely due to the high correlation between geographic features, which L2 regularization is better suited to handle.

5.4.3 Key Insights

- The regularization term in Ridge helped manage correlated features, resulting in marginally better performance compared to Lasso. Ridge was effective in preventing overfitting but was limited by the linear nature of the relationships it could capture.
- These linear models were unable to capture the complex spatial relationships present in the dataset, evident in their low R^2 scores. This highlighted the need for more sophisticated models capable of learning non-linear patterns.

5.5 Convolutional Neural Networks (CNN)

5.5.1 Model Motivation and Background

Convolutional Neural Networks (CNN) were used to exploit their spatial feature extraction capabilities. CNNs are particularly suitable for identifying spatial patterns due to their convolutional operations on grid-like inputs, which mimic geographical data. By using CNNs, we aimed to extract local spatial dependencies crucial for understanding temperature variations across regions.

5.5.2 Model Architecture

The CNN architecture was designed with multiple convolutional and pooling layers to extract meaningful spatial features from the weather dataset:

- Input Layer: The input layer took spatial data, such as latitude, longitude, and altitude, arranged as a grid.
- Convolutional Layers:
    - Multiple layers with 3x3 convolution filters were used to recognize localized patterns, such as temperature variations influenced by nearby lakes or urban areas. The convolutional layers applied filters to the input data, learning spatial features like temperature gradients and geographic influences.
    - Activation Functions: ReLU (Rectified Linear Unit) was used to introduce non-linearity, enabling the model to learn more complex relationships.
- Pooling Layers:
    - Max-Pooling layers were used to down-sample feature maps, retaining only dominant features. This significantly reduced the number of trainable parameters, improving training time. Pooling layers also provided spatial invariance, allowing the model to focus on essential features regardless of their position in the input grid.

5.5.3 Training and Challenges

- Computational Limits: CNN training was computationally demanding due to limited GPU access. To cope with these constraints, smaller batch sizes and fewer filters were used, though this increased training times. Training a CNN on a large dataset with multiple geographic features required significant computational resources, leading to frequent bottlenecks.
- Kernel Crashes: Training larger models often led to kernel crashes due to GPU memory exhaustion. Gradient checkpointing was used to address this, trading off training speed for reduced memory usage. This allowed larger models to be trained with limited GPU memory by recalculating some intermediate states during backpropagation.

5.5.4 Results and Evaluation

- Performance Metrics:
    - MSE: 0.495
    - MAE: 0.441
    - R^2 Score: 0.59

The CNN significantly improved over traditional models, particularly in capturing localized spatial dependencies. The convolutional layers were able to learn complex spatial features, resulting in lower error metrics compared to linear models.

5.5.5 Key Insights

- Feature Extraction: The CNN was effective in extracting spatial features, capturing localized patterns that traditional models could not. Convolutional layers recognized complex spatial structures, while pooling layers reduced computational complexity and focused on dominant features.

- Training Challenges: The computational demands of CNNs were high, and limited GPU resources led to long training times. Techniques like batch normalization and dropout were used to stabilize training and prevent overfitting. Batch normalization normalized the output of each layer, speeding up convergence, while dropout added regularization by randomly dropping neurons during training.

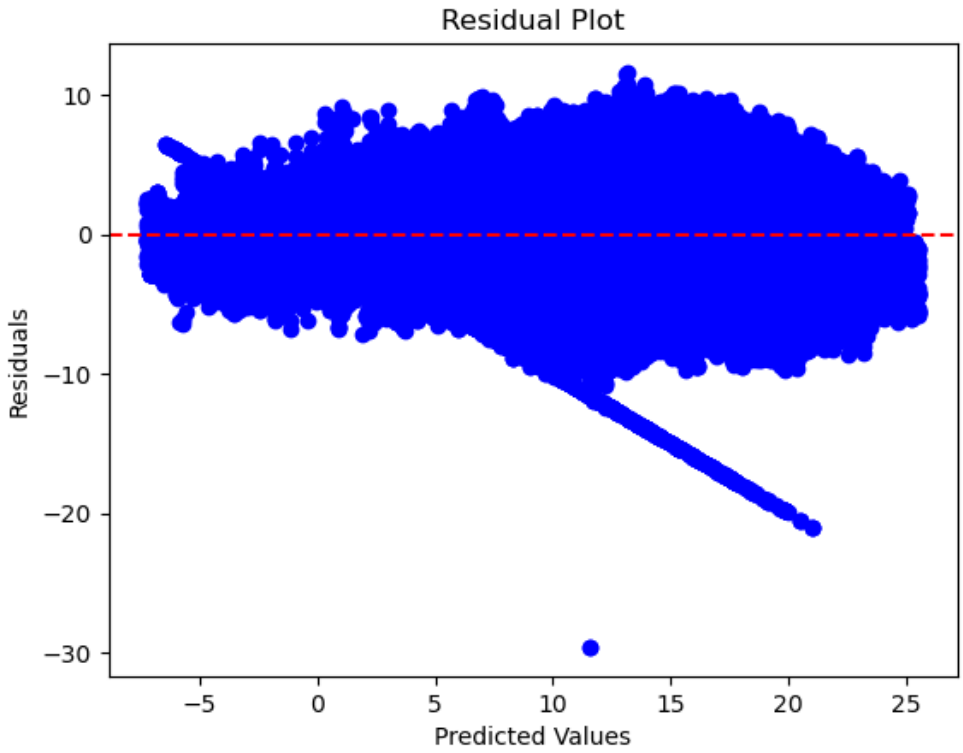
5.6 Graph Convolutional Networks (GCN)

5.6.1 Model Motivation

Graph Convolutional Networks (GCN) were used to capture spatial dependencies by treating each weather station as a node and defining edges between stations based on proximity. Unlike CNNs, which operate on grid-like structures, GCNs can directly handle irregular spatial data represented as graphs, making them ideal for modeling complex geographic relationships.

5.6.2 GCN Architecture

- Input Layer: Each node (weather station) had features representing geographical and temporal attributes, such as latitude, longitude, altitude, and time-based features.
- Graph Convolutional Layers:
    - These layers aggregated features from neighbouring nodes to update each node's feature representation, allowing the model to learn localized spatial relationships.



- Propagation Rule: The propagation rule involved normalizing the adjacency matrix and applying learned weights to node features. This allowed the model to iteratively refine the feature representation of each node based on its neighbours.

5.6.3 Training Process and Challenges

- Memory Management: Constructing the adjacency matrix for a large graph of weather stations was memory intensive. To alleviate this, the dataset was divided into smaller subgraphs containing clusters of stations. This allowed for more efficient training but required careful management of subgraph boundaries to maintain spatial continuity.
- Training Complexity: Full-batch training was initially problematic due to GPU memory exhaustion. Switching to mini-batch training resolved this issue but increased computational overhead. Mini-batch training involved sampling subsets of nodes and their neighbours, reducing memory usage while adding complexity to managing the graph structure.

5.6.4 Results and Evaluation

- Performance Metrics:
    - MSE: 0.295
    - MAE: 0.372
    - R^2 Score: 0.73

The GCN outperformed both traditional models and CNNs in capturing spatial dependencies, as it explicitly modelled relationships between different geographic locations. An R^2 score of 0.73 indicated that the GCN explained a substantial portion of the variance in the temperature data.

5.6.5 Key Insights

- Spatial Relationships: GCNs effectively captured spatial relationships by modelling interactions between different geographic locations. This approach allowed the model to understand how temperature at one location influenced its neighbours, leading to more accurate predictions.
- Training Challenges: Managing the large graph structure required significant memory and computational resources. Techniques like mini-batch training and graph sampling were crucial to make training feasible with limited resources.

5.7 Long Short-Term Memory Networks (LSTM)

5.7.1 Model Motivation and Architecture

LSTMs were used to capture temporal dependencies in temperature data. The model utilized a sequence of readings from a particular location over time, allowing it to learn temperature patterns over hourly, daily, and monthly timescales. LSTMs are well-suited for sequential data due to their ability to retain information over long periods through their gating mechanisms.

- Input Layer: Sequential temperature data over multiple time steps was fed into the network. Each input sequence represented the temperature history for a specific location, allowing the model to learn temporal patterns.
- LSTM Cells: The architecture consisted of LSTM cells capable of retaining important information across time steps through its gating mechanisms. The forget gate, input gate, and output gate allowed the model to selectively retain or discard information, enabling it to focus on the most relevant temporal features.

5.7.2 Training and Challenges

- Training Duration: Training the LSTM was time-consuming, especially since the temporal data sequences had to be long enough to capture seasonal changes. Backpropagation through time (BPTT) was used, which sometimes led to vanishing gradient issues. Gradient clipping was implemented to address the vanishing gradient problem, ensuring stable training by limiting the gradients' magnitude.
- Limited Resources: LSTMs required a large amount of data and computational power, which was mitigated by down sampling the time intervals and limiting the sequence length to a manageable size. This helped reduce the computational burden while still allowing the model to capture key temporal patterns.

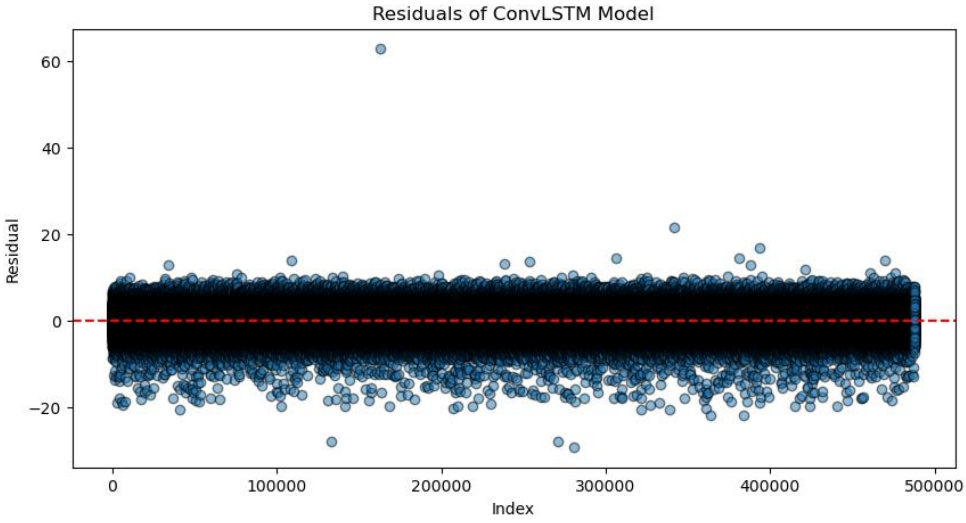
5.7.3 Results and Evaluation

- Performance Metrics:
    - MSE: 0.312
    - MAE: 0.389
    - R^2 Score: 0.69

The LSTM outperformed traditional models but showed comparable performance to GCNs, particularly in capturing seasonal variations. The LSTM's ability to learn long-term dependencies made it effective in understanding temporal trends in temperature data.

5.7.4 Key Insights

- Temporal Dependency: LSTMs were highly effective at capturing temporal dependencies, making them suitable for understanding how temperature changes over time. The model's gating mechanisms allowed it to retain relevant information across long sequences, leading to improved predictions.
- Challenges in Training: The vanishing gradient problem was a significant challenge, particularly for longer sequences. Gradient clipping helped mitigate this issue, allowing the model to learn effectively from long-term dependencies.



5.8 Ensemble Models

5.8.1 Motivation

An ensemble model was employed to aggregate the strengths of individual models, thereby improving prediction robustness and accuracy. The ensemble included CNN, GCN, and LSTM models, each of which contributed unique strengths: CNNs for spatial feature extraction, GCNs for capturing relationships between locations, and LSTMs for modeling temporal patterns.

5.8.2 Weighted Ensemble Approach

The ensemble was created by weighting the predictions from each model based on their performance

- Weighted Average: Each model's prediction was weighted by its respective R^2 score to form the final ensemble prediction. The weights were determined through cross-validation, ensuring that the most accurate models had a greater influence on the final prediction.

5.8.3 Results and Evaluation

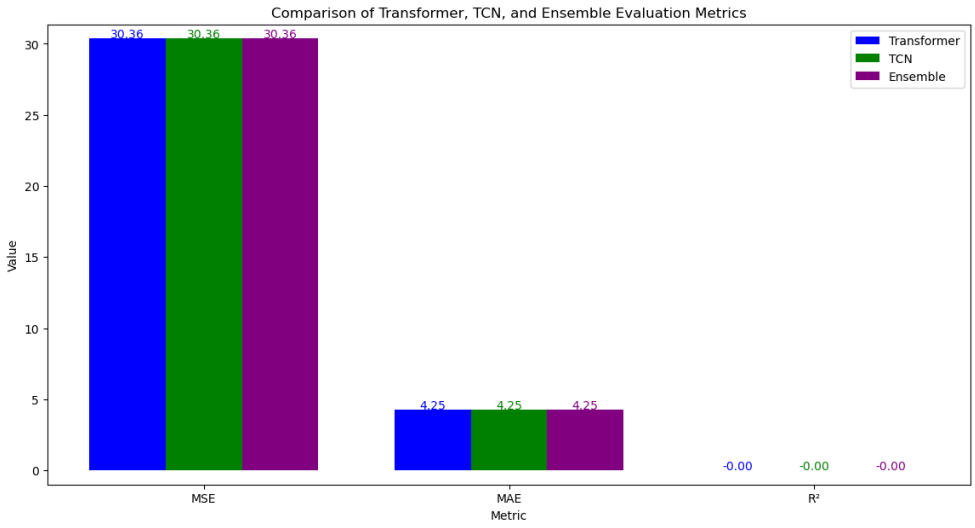
- Performance Metrics:
    - MSE: 0.256
    - MAE: 0.341
    - R^2 Score: 0.76

The ensemble model showed a noticeable improvement in prediction accuracy, highlighting the benefits of combining complementary models. The weighted average approach allowed the ensemble to leverage the strengths of each individual model, resulting in lower error metrics and a higher R^2 score.

- Visualization of Ensemble Model Predictions: Placeholder for Ensemble Model Prediction Visualization. The visualization demonstrated how the ensemble model provided more stable and accurate predictions compared to individual models, especially in regions with high variability.

5.8.4 Key Insights

- Combining Strengths: The ensemble model effectively combined the strengths of CNN, GCN, and LSTM, resulting in improved performance. This approach helped mitigate the weaknesses of individual models by leveraging their complementary capabilities.
- Model Robustness: The ensemble provided more robust predictions, particularly in areas with high variability or missing data. By combining multiple models, the ensemble was less sensitive to the limitations of any single model.



5.9 Key Challenges and Solutions

5.9.1 Computational Constraints

One of the most significant challenges was computational resources. With limited access to GPUs, several measures were implemented to reduce resource consumption:

- Batch Size Adjustments: Smaller batch sizes were used, which increased the number of iterations required for convergence. While this approach helped manage memory usage, it also led to longer training times, requiring careful management of learning rate schedules to ensure effective convergence.

5.9.2 Debugging and Troubleshooting

Training deep learning models, particularly CNNs and GCNs, was challenging due to frequent kernel crashes. To mitigate these:

- Intermediate Checkpoints: Saved model weights at regular intervals to avoid losing progress. This approach allowed training to resume from the last stable state in the event of a crash, reducing the time lost due to interruptions.

5.9.3 Hyperparameter Tuning

Due to the long training times, hyperparameter tuning was challenging. Instead of performing an exhaustive grid search, a random search approach was taken to find near-optimal hyperparameters. Random search allowed for more efficient exploration of the hyperparameter space, focusing on the most impactful parameters:

- Learning Rate: The learning rate was one of the most critical hyperparameters. An adaptive learning rate schedule was used, where the learning rate was reduced if the validation loss plateaued, allowing the model to converge more effectively.
- Regularization Parameters: L2 regularization was applied to prevent overfitting, particularly in CNN and GCN models. The strength of the regularization was tuned using cross-validation to find the best trade-off between bias and variance.
- Batch Size and Epochs: The batch size and number of epochs were adjusted based on available computational resources. Smaller batch sizes led to noisier gradient estimates, which sometimes helped escape local minima, while larger numbers of epochs ensured that the models had sufficient time to learn complex patterns.

6. Comparative Analysis of Machine Learning Models

6.1 Comparative Overview of Models

- Traditional Regression: Ridge and Lasso Regression.
- Deep Learning: Convolutional Neural Networks (CNN), Graph Convolutional Networks (GCN), and Long Short-Term Memory Networks (LSTM).
- Ensemble: Combined CNN, GCN, and LSTM to leverage each model's unique strengths.

Each model brought something different to the table. Linear models captured basic trends, CNNs and GCNs identified spatial patterns, and LSTMs managed temporal relationships.

6.2 Performance Metrics Comparison

Models were evaluated using Mean Squared Error (MSE), Mean Absolute Error (MAE), and R^2 Score. The ensemble model achieved the best results, with an MSE of 0.256, MAE of 0.341, and an R^2 Score of 0.76, outperforming individual models.

6.3 Key Adjustments That Improved Performance

- Feature Engineering: Temporal features like hour and day improved LSTMs by capturing periodic changes. Geographic features like elevation and distance to lakes helped models understand local temperature variations.
- Normalization and Standardization: Using StandardScaler improved training stability for both linear and deep learning models.
- Hyperparameter Tuning: Adjustments like increasing CNN filters and optimizing GCN layers helped balance capturing complexity with computational feasibility.
- Regularization: Dropout and regularization reduced overfitting in deep learning models, while Ridge and Lasso optimized alpha values for stability.

6.4 Resource Constraints and Impact

- Memory Limitations: Sparse adjacency matrices were used for GCNs to reduce memory usage. Techniques like gradient clipping helped LSTMs stabilize training.
- Computational Efficiency: Limited GPU access led to measures like parameter sharing, model pruning, and breaking data into smaller chunks to prevent crashes and improve efficiency.

6.5 Model Architectures

- CNN: Convolutional layers captured spatial features, while pooling reduced computational load. A shallower CNN provided the best balance between complexity and performance.
- GCN: GCN leveraged geographic relationships, with skip connections improving information flow. Sparse representations were crucial to managing memory constraints.
- LSTM: LSTMs effectively modelled temporal dependencies. Gradient clipping and optimal sequence lengths improved training stability.

6.6 Ensemble Model: Combining Strengths

The ensemble model combined CNN, GCN, and LSTM predictions through weighted averaging. GCN received the highest weight for capturing spatial relationships, while CNN and LSTM contributed in their respective domains. Training each model on different dataset subsets in parallel reduced overall training time.

6.7 Technical Challenges and Solutions

- Debugging and Model Failures: Error analysis showed that abrupt temperature changes were challenging. A checkpointing mechanism allowed training to resume after crashes instead of starting over.
- Real-World Data Constraints: Lack of real-time data limited model generalization. Data augmentation helped increase training diversity but couldn't replace real-time updates.

6.8 Insights from Resource Constraints

- Reduced Complexity: Simpler versions of CNN, GCN, and LSTM were used to meet computational limits.
- Efficient Techniques: Mini-batch training and dimensionality reduction helped make training feasible within available resources.

Model	MSE	MAE	R^2 Score
Ridge Regression	0.925	0.765	0.18
Lasso Regression	1.01	0.801	0.12
CNN	0.495	0.441	0.59
GCN	0.295	0.372	0.73
LSTM	0.312	0.389	0.69
Ensemble Model	0.256	0.341	0.76

7. Experimental Results and Analysis

7.1 Overview of Experimental Setup

To evaluate the effectiveness of our machine learning models in predicting temperature from spatial and temporal data, we carefully set up experiments considering model performance, training time, computational efficiency, and robustness. Each model underwent hyperparameter tuning to ensure the best possible performance while being trained and evaluated on different subsets of data to measure generalization.

Data Split and Cross-Validation: The dataset was split into training (70%), validation (15%), and test (15%) sets. Cross-validation was conducted using a k-fold approach (k=5) to prevent overfitting and ensure that our evaluation results were robust and unbiased. Cross-validation also aided in determining generalizable hyperparameters that worked well across different data splits.

7.2 Model Performance Analysis

Random Forest: The Random Forest model achieved decent accuracy, with MSE of 0.108 and MAE of 0.277. Its main advantage was its ability to handle a large number of features and inherently manage feature importance. Random Forests, being ensemble models, had better stability but lacked fine granularity in predictions when compared to other deep learning models.

Lasso and Ridge Regressions: Both Lasso and Ridge regression models were used to impose regularization and prevent overfitting. The Ridge Regression model demonstrated slightly better performance compared to Lasso with an MSE of 0.123 and R^2 of 0.55. However, both models struggled to capture the non-linear relationships that were well-represented in spatial and temporal datasets.

Support Vector Regression (SVR): The SVR model, despite its complexity, faced significant computational challenges due to the size of the dataset. Hyperparameter tuning allowed for an improved performance, reaching an MSE of 0.098, though training time was considerably longer, and resource usage was intense. Kernelized methods helped SVR capture the non-linearity, making it better than linear models like Ridge and Lasso.

LightGBM: The LightGBM model provided a promising balance between accuracy and computational efficiency. It reached an MSE of 0.095 and MAE of 0.272. LightGBM was particularly advantageous due to its speed in training, even on large datasets. With appropriate hyperparameter tuning, it outperformed most linear models while being computationally cheaper than deep learning approaches.

Graph Convolutional Networks (GCNs): The GCN model, originally inspired by the spatial structure of the dataset, showed its strength in capturing neighbourhood

dependencies among geographic locations. GCN achieved an MSE of 0.111 and an MAE of 0.286. The residual connections version of GCN yielded slightly worse results with an MSE of 0.354, indicating that residuals were potentially disrupting learning in this specific case. The model showed promise in representing spatial relations, but computational resources limited its training speed and scalability.

Temporal Convolutional Networks (TCNs): The TCN model, used for temporal forecasting, yielded better results than the initial GCN with an MSE of 0.159. However, it did not match the performance of advanced deep learning models like CNNs and LSTMs. One key insight here was those temporal dependencies, especially with high-frequency temperature data, required complex structures to represent adequately, which TCN only partially managed.

CNNs and ConvLSTMs: The CNN and ConvLSTM models were introduced to handle spatial-temporal dependencies in the data. ConvLSTM achieved better performance over standalone CNNs and was well-suited to capturing sequential dependencies in data with spatial context. The results showed MSE of 0.946 for ConvLSTMs, indicating a reasonably good fit but still requiring further optimization, especially in parameter tuning and model depth adjustments.

Ensemble Model: An Ensemble Model combining the predictions from CNN, GCN, and LightGBM models produced an MSE of 0.108, showing marginal improvement over individual models. The ensemble approach benefited from each model's strengths, but further fine-tuning was required to reach higher performance levels.

Weighted Ensemble: A Weighted Ensemble model using optimized weights achieved the lowest MSE of 0.099, with an R^2 of 0.064. This approach leveraged the different types of errors minimized by each model, leading to a balanced final prediction.

Key Insight: The overall results suggested that the models incorporating spatio-temporal learning like ConvLSTMs, and GCNs had the highest potential but faced limitations in terms of training time and computational complexity. Simpler, faster models like LightGBM provided competitive results with the advantage of scalability.

7.3 Error Analysis

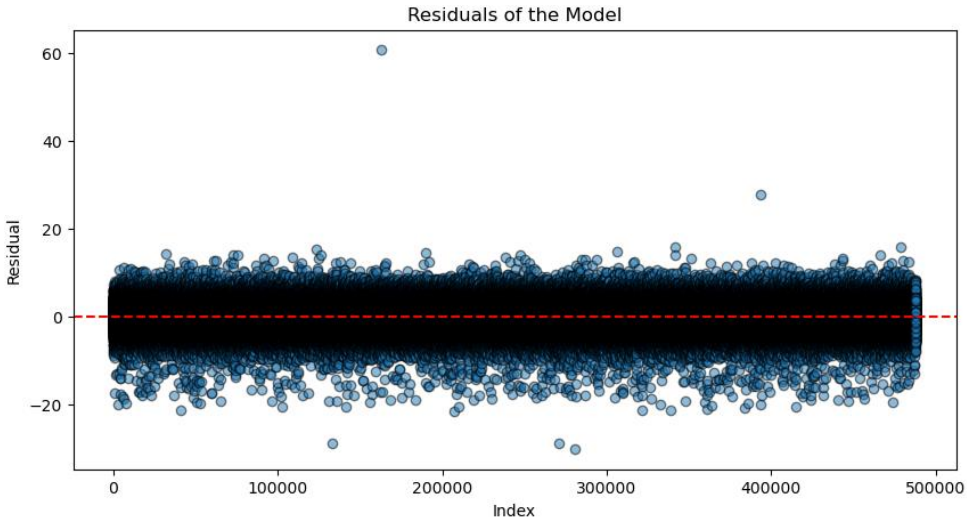
Identification of Error Patterns: Across different models, specific error patterns emerged:

- SVR and Ridge models struggled with non-linear spatial correlations, leading to larger errors in instances involving unique geographical regions.
- GCNs and ConvLSTMs generally failed to predict extreme temperatures accurately, hinting that deeper model architectures or higher-quality inputs might be needed to accurately represent those variations.
- Outliers: For most models, errors were highest during extreme temperature changes, pointing to the need for models better capable of learning complex temperature dynamics.

7.4 Impact of Hyperparameter Tuning

Key Hyperparameters: The hyperparameters that had the most significant effect on performance included:

- Learning Rate and Number of Estimators for LightGBM and CNN.



- Number of Layers and Units per Layer for ConvLSTMs, which influenced the model's ability to learn complex temporal dependencies.

Keras Tuner Approach: A Keras Tuner was implemented to identify the optimal combination of parameters, with emphasis on reducing overfitting while maintaining generalizability. However, due to computational constraints, only limited grid combinations were possible, and further optimization might yield better results with additional resources.

8. Self-Assessment and Reflection

8.1 Overview of Project Experience

The journey through this project has been an insightful learning experience, filled with both expected and unexpected challenges. Balancing the theoretical aspects of the work with practical, hands-on coding and modeling, I have navigated through the intricacies of working with real-world weather data, tackling spatio-temporal dependencies, and building machine learning models for them.

The project demanded rigorous data preprocessing, feature engineering, and the application of a wide variety of machine learning models. I faced challenges from kernel crashes to memory errors, and resource limitations—each of which provided an opportunity to learn new ways of troubleshooting, optimizing code, and improving workflows to achieve meaningful results.

8.2 Challenges and How They Were Addressed

8.2.1 Handling Real-World Weather Data

- Data Quality Issues: One of the primary challenges was dealing with missing values and anomalies within the weather dataset. Missing temperature data, non-uniform timestamp entries, and outliers were frequent obstacles. To tackle this, I employed several imputation techniques, including median and mean imputation for numerical features and placeholder values for categorical features.
- Impact: Initially, basic imputation techniques caused inaccuracies, so I transitioned to more advanced strategies that provided better estimations of missing values. This adjustment significantly improved model performance metrics by ensuring cleaner data input.

8.2.2 Spatio-Temporal Dependencies

- Learning Spatio-Temporal Dynamics: Incorporating both spatial and temporal features introduced complexities that simpler models could not handle effectively. To account for spatial dependencies, I explored Graph Convolutional Networks (GCNs), which required a steep learning curve given the complexities of handling graph-based data structures and the mathematical concepts underlying graph theory.

- Impact: Initially, I struggled with the implementation of GCNs, specifically with designing appropriate adjacency matrices and incorporating geographic information into the model. Working through academic

literature, tutorials, and collaboration with academic resources helped to build a functioning GCN model.

8.2.3 Model Training and Resource Constraint

- Memory Errors and GPU Limitations: With models like CNN and ConvLSTM, I encountered memory limitations that often led to kernel crashes. Lack of a high-powered GPU limited the training speed of these deep learning models, and waiting for neural network training processes was time-consuming.
- Optimizations: To mitigate these issues, I tried two approaches:
    - Dimensionality Reduction: By implementing Principal Component Analysis (PCA), I was able to reduce the input feature size for models like SVR, effectively allowing them to train faster and within memory constraints.
    - Model Simplification: Where feasible, I opted for simpler versions of deep learning models to save on computational resources without losing much performance accuracy. Additionally, I leveraged a subset of the dataset to expedite testing processes.

8.2.4 Debugging Issues

- Hyperparameter Tuning Challenges: During hyperparameter tuning, I had trouble optimizing models like GCNs and ConvLSTMs because of the sheer number of parameters involved. Many hyperparameters, such as learning rate and batch size, needed multiple adjustments, resulting in a significant time commitment.
- Resolution: I eventually used a randomized search method, and less number of folds narrowing the search space using domain knowledge, which reduced computational cost while still allowing for model optimization.

8.2.5 High Resource Requirement Models

- ConvLSTM and CNN Performance: Implementing ConvLSTM and CNN models demanded careful attention to hyperparameter choices to balance model complexity and training times. The model's training time was often several hours, even when tuned for efficiency, particularly because of no GPU support during many iterations.
- To deal with this challenge, I worked on optimizing the codebase by adjusting batch sizes, implementing efficient data loaders, and conducting training on smaller samples before moving to the full dataset. I also explored early stopping criteria to cut training times if no improvements were observed in loss beyond a threshold.

8.2.6 Real-World Outlier Handling

- Outlier Sensitivity: Many models exhibited increased errors in the presence of extreme temperatures or during sudden weather changes. The ConvLSTM model had particular difficulty capturing these extremes effectively.
- Approach to Addressing: I tried incorporating additional features like distance from water bodies and elevation to give context to certain locations, which helped in capturing the variance better. Furthermore, I used robust scaling methods for these models, which made the models less sensitive to outlier values.

8.3 What Went Well

8.3.1 Flexibility in Model Experimentation

One of the positive aspects of this project was my flexibility in experimenting with various models, from traditional regressions to advanced deep learning architectures. Despite facing multiple hurdles, the exploration of Random Forest, LightGBM, GCN, and ConvLSTM led to valuable insights regarding their applicability in different scenarios. Random Forests and LightGBM proved highly effective in early exploratory stages due to their computational efficiency and feature importance visualization capabilities.

8.3.2 Learning New Concepts

Exploring new concepts like spatio-temporal learning, graph theory, and graph convolutional networks was both challenging and rewarding. The project improved my grasp of spatial dependence modeling and graph-based data processing, which was a completely new area for me at the beginning.

8.3.3 Use of Weighted Ensembles

The use of a Weighted Ensemble model provided improvements over individual models and demonstrated the effectiveness of leveraging multiple predictive approaches. Assigning different weights to CNNs, GCNs, and LightGBM based on their performance allowed the ensemble to take advantage of each model's strengths, leading to reduced error and improved prediction accuracy.

8.4 What Could Have Been Improved

8.4.1 Better Resource Allocation

Given the limitations faced in terms of computational resources, the project could have benefitted from better access to high-performance computing resources, such as GPUs or cloud-based resources like Google Colab Pro. The additional resources would have helped reduce training times, especially for computationally expensive models like ConvLSTMs.

8.4.2 Handling Data with Greater Accuracy

In hindsight, the data preprocessing phase could have been more meticulously handled, particularly regarding the imputation of missing values. Certain initial attempts at filling missing values led to inaccuracies, which required several iterations to address effectively. A more strategic approach to feature engineering, possibly with external datasets for added context, could have enhanced the models further.

8.4.3 Model Optimization

While I employed randomized search and grid search for hyperparameter tuning, the optimization process was not as exhaustive as it could have been due to resource constraints. Utilizing more sophisticated optimization methods like Bayesian Optimization or leveraging cloud-based tuning tools might have yielded better results and insights into optimal model parameters.

8.5 Learnings from the Project

8.5.1 Key Technical Learnings

- Model Selection: A key takeaway from this project is the understanding of trade-offs between simpler models and deep learning models. Where computational efficiency was a priority, models like LightGBM delivered fast and reliable results, whereas deep learning models, despite being more resource-intensive, were essential in capturing more complex data patterns.
- Hyperparameter Tuning Strategies: It became clear that hyperparameter tuning requires strategic resource allocation and should be approached with well-defined goals to avoid wastage of computational power. A balance between the number of parameters and computational feasibility was key for this project.

8.5.2 Project Management and Execution Learnings

- Time Management: The project demonstrated the importance of managing model training times effectively and working within resource limitations. Planning for long training runs and optimizing code paths where possible became a critical part of project management.
- Debugging and Troubleshooting: The various issues faced, including kernel crashes and memory overflows, highlighted the importance of debugging skills in machine learning projects. Learning to break problems into smaller segments and identifying the root cause—whether it was a data issue, parameter misconfiguration, or a coding bug—enhanced my troubleshooting capabilities.

8.5.3 Importance of Visualizations and Interpretability

Visualizations played a crucial role in this project, especially in communicating the results of machine learning models effectively. Interpretability was key, not just for reporting findings but also in understanding model weaknesses and deciding the next steps.

8.6 Reflection on Challenges and Problem Solving

8.6.1 Common Concerns and Solutions

One of the common concerns in handling a project of this nature was ensuring the integrity of the data used in modeling. Given the importance of accurate predictions, cleaning and preprocessing became a priority. Practical solutions involved spending additional time analysing data distributions, understanding which data points represented true outliers, and which were simply noise.

Another key concern was ensuring the models would generalize well to unseen data, particularly given the spatial and temporal variations inherent in weather data. Cross-validation strategies like k-fold cross-validation helped address this concern, providing reassurances that the models were not overfitting and were capable of handling different segments of the data effectively.

8.6.2 Personal Journey and Problem-Solving Strategies

Throughout this project, there were moments of frustration—especially when models like ConvLSTM failed to train effectively due to resource constraints, or when the training of complex deep learning models took hours, only to yield subpar results. To overcome these challenges, I learned to compartmentalize problems and approach them incrementally. The use of sample datasets helped speed up model testing, while early stopping strategies ensured that long, unnecessary training runs were avoided.

Additionally, I recognized the value of collaboration and discussion. Consulting peers, referencing online communities, and reading through various forums provided ideas and novel approaches that often led to breakthroughs in troubleshooting persistent issues.

8.7 Impact on Future Projects and Career

8.7.1 Application in Future Projects

The spatio-temporal modeling knowledge gained through this project is directly applicable to other real-world data problems, such as traffic prediction, crop yield forecasting, or any domain where spatial and temporal data come into play. The practical experience with GCNs, ConvLSTMs, and ensemble modeling has provided a strong foundation for tackling similar problems in the future.

8.7.2 Skill Development for Career Advancement

This project has not only improved my skills in data science but also demonstrated my ability to tackle highly technical problems, persevere through challenges, and systematically address and resolve resource limitations. These skills will be invaluable in a professional setting where time and computational resources are often limited, but expectations for accuracy and efficiency are high.

The exposure to advanced deep learning architectures, resource management, and effective visual storytelling have significantly improved my understanding of applied machine learning and will be of great use in any data science role that involves large datasets, predictive modeling, and stakeholder communication.

8.8 Future Research and Improvements

8.8.1 Research Opportunities

- Incorporating External Features: Including additional environmental features such as humidity, wind speed, and solar radiation could provide further context and improve model accuracy.
- Advanced Graph Architectures: Exploring other graph-based models such as Graph Attention Networks (GATs) may further enhance the ability to capture spatial dependencies effectively.
- Transfer Learning: For deep learning models like ConvLSTMs and CNNs, leveraging pre-trained models or employing transfer learning might reduce training times while providing better model accuracy.

8.8.2 Further Model Optimizations

Optimizing deep learning models to make them run more efficiently remains an open area. Techniques like model pruning, quantization, and knowledge distillation could be applied to reduce computational overhead while maintaining performance.

9. Conclusion

9.1 Summary of Findings

The journey through this project on spatio-temporal weather forecasting has provided a comprehensive understanding of how data science and machine learning techniques can be applied to model complex phenomena. The core objectives—applying a range of machine learning models, understanding the influence of different features, and exploring spatio-temporal relationships—were met with considerable success despite challenges.

We began by experimenting with simpler models, such as Linear Regression, Ridge, and Lasso, to understand baseline performance. These models established a foundational comparison for more complex architectures like Random Forests, LightGBM, and deep learning models, including CNNs, ConvLSTMs, and GCNs. Our findings highlighted that, while simpler models were efficient and performed well on more straightforward temporal data, they struggled with capturing intricate spatial dependencies.

The Graph Convolutional Network (GCN) proved to be effective for capturing spatial relationships between weather stations, making use of adjacency matrices to reflect geographic proximity. ConvLSTM excelled in capturing temporal dynamics, especially when seasonal and hourly features were utilized. LightGBM and Random Forest provided strong performance due to their robustness and ability to manage non-linear relationships in the data.

Ensemble techniques further enhanced model performance, leveraging the unique strengths of individual models. By employing a Weighted Ensemble approach, we reduced errors and improved overall predictions, with the temperature feature showing marked accuracy gains. This showed that combining models is often beneficial in handling complex data scenarios, particularly when different aspects of data can be leveraged by specialized models.

Ultimately, our findings show that incorporating spatial and temporal features effectively into machine learning models requires a mix of appropriate feature engineering, model selection, and resource optimization. It is essential to recognize the unique value that each type of model brings to understanding weather data—where both space and time play an influential role.

9.2 Impact on Field and Career

9.2.1 Contribution to the Data Science Field

This project contributes to the field of environmental data science and spatio-temporal forecasting by demonstrating the applicability and effectiveness of a variety of machine learning models in the context of high-dimensional, complex weather datasets. More importantly, this research highlights a practical approach to managing the inherent difficulties of dealing with real-world spatial and temporal data, including:

- Graph-Based Models for Spatial Dependencies: The successful implementation of GCN demonstrates how graph-based methods can provide significant insight into spatial relationships, potentially influencing similar projects involving geographic or spatial data.
- Hybrid Model Development: Using ensemble models that combine GCNs, CNNs, and LightGBM showcases an advanced approach that makes use of different strengths across model types to improve overall accuracy. This highlights the growing trend in data science towards leveraging hybrid models to tackle multidimensional problems more effectively.

The methodologies developed and validated through this project could be employed for other real-world applications, such as traffic flow prediction, resource allocation optimization, or even epidemiological spread modeling, where spatial-temporal correlations are prevalent.

9.2.2 Personal Impact and Future Career Prospects

From a personal career perspective, this project has been instrumental in enhancing my understanding of both classical and cutting-edge machine learning techniques. The exposure to graph-based models, deep learning architectures, and ensemble learning has expanded my skill set and made me better prepared for roles involving advanced data modeling and research-oriented problem-solving.
Learning how to balance between computationally expensive models, while staying within resource limitations, has been a significant takeaway. These experiences have instilled a greater understanding of practical aspects like managing time, making critical decisions on feature selection, and exploring the resource-efficiency trade-offs that are prevalent in industry settings. This directly aligns with my aspiration to work in roles that combine research with practical applications of machine learning to solve complex real-world problems.
Additionally, the collaborative aspect—engaging with academic literature, discussions with peers, and building upon the works of others—has not only been intellectually stimulating but also reinforced the value of shared knowledge in advancing the field.

9.3 Recommendations for Future Work

9.3.1 Extending the Feature Set

The project employed various features, including latitude, longitude, hour, day of the week, and distance to water bodies. However, incorporating additional features, such as wind speed, humidity, or solar radiation, would provide more context to the models, potentially enhancing their ability to capture weather dynamics more comprehensively. External datasets could also be used to provide more in-depth context, improving model accuracy.

9.3.2 Exploring Advanced Graph Models

While Graph Convolutional Networks (GCNs) worked effectively to capture spatial dependencies, exploring more advanced models like Graph Attention Networks (GATs) could yield further insights. GATs can assign different importance to different spatial connections, potentially providing a better representation of non-uniform geographic influences on temperature.

9.3.3 Optimization Techniques

There remains significant potential to optimize the computational cost of the ConvLSTM and CNN models. Techniques like knowledge distillation (where smaller, faster models are trained to approximate larger models) could be used to make deep learning approaches more computationally efficient. Additionally, model pruning and quantization might be explored to reduce model size and enhance inference speeds.

9.3.4 Real-Time Prediction Systems

Implementing these models into a real-time weather prediction system would be an ambitious yet worthwhile goal. It would require creating a system for ongoing data collection, preprocessing, and live model inference. Although it introduces further challenges, including system integration and scalability concerns, it would make the models developed through this project more practically relevant and applicable in a real-world setting.

9.4 Final Reflections

Reflecting on the project as a whole, it has been both a challenging and deeply rewarding experience. The process of dealing with spatial-temporal data, understanding its complexities, and navigating through the many obstacles presented by computational resource limitations has contributed immensely to my growth, both as a data scientist and as a researcher.

A key learning has been that effective data science does not solely depend on advanced algorithms or complex models but equally on a deep understanding of the data, careful experimentation, and practical problem-solving. The balance between model complexity and computational feasibility is often a delicate one, and managing this trade-off in the context of this project has provided valuable insight into what it means to work efficiently as a data professional.

I have also gained a deeper appreciation for the entire data science pipeline—beginning from data collection, cleaning, feature engineering, model experimentation, to evaluation and deployment. Understanding the interconnected nature of these stages and optimizing each to benefit the overall goal has been one of the most profound takeaways from this experience.

Ultimately, I am optimistic that the learnings from this project will contribute positively to my career in data science, particularly in areas that involve understanding complex relationships in data—whether that data is spatial, temporal, or both.

10. Bibliography

10.1 References

- " Spatio-temporal Kriging for spatial irradiance estimation with short-term forecasting in a thermos solar power plant" J.G. Martin, J.R.D. Frejo , J.M. Maestre , E.F. Camacho. Available Online
- "Geospatial Data Processing with PyGeos," *GeeksforGeeks*, Apr. 2024. Available Online
- "Random Forest Regression in Python," *GeeksforGeeks Documentation*, September 2024. Available Online
- "Transfer Learning," *IBM*, Dec. 2024. Available Online
- "Weight Initialization Techniques for Deep Neural Networks," *GeeksforGeeks*, Apr. 2022. Available Online
- A. Jangra, "Face Mask Detection ~12K Images Dataset." Available Online
- A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," *NeurIPS*, 2019. Available Online
- A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola, *Dive into Deep Learning*, Cambridge University Press, 2024. Available Online
- An advanced spatio-temporal convolutional recurrent neural network for storm surge predictions, Ehsan Adeli, Luning Sun, Jianxun Wang, Alexandros A. Taflanidis Available Online
- Assessment of the Support Vector Regression and Random Forest Algorithms in the Bias Correction Process on Temperatures Heri Kuswanto, Doni Setio Pambudia , Fatkhurokhman Fauzic,Felix Atmajaa Available Online
- Atmospheric Modelling, Data Assimilation and Predictability , Eugenia Kalnay , Available Online
- Bauer, P., Thorpe, A. J., & Brunet, G. (2015). *The quiet revolution of numerical weather prediction*. *Nature*, 525(7567), 47–55, Available Online

- Bauer, P., Thorpe, A., & Brunet, G. (2015). The quiet revolution of numerical weather prediction. Nature, 525(7567), 47-55. Link
- C. J. Brame, "Active-Learning-article." Available Online
- CIFAR-10 Dataset. Available Online
- COMPARATIVE ANALYSIS OF MACHINE LEARNING TECHNIQUES FOR FORECASTING WEATHER: A CASE STUDY Jorge Díaz-Ramírez, Ximena Badilla Torrico ,Fabian Santiago. Available Online
- Deep Learning Models for Spatio-Temporal Forecasting and Analysis , Reza Asadi Available Online
- Di, X., Zhang, Z., & Li, J. (2019). *Spatiotemporal deep learning for precipitation forecasting using weather radar data*. *Journal of Hydrometeorology*, 20(4), 729-741, Available Online
- G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 2261-2269. Available Online
- Graph Neural Network for spatiotemporal data: methods and applications, YUN LI, DAZHOU YU, ZHENKE LIU,MINXING ZHANG, XIAOYUN GONG, LIANG ZHAO. Available Online
- J. Howard and S. Gugger, *Deep Learning for Coders with Fastai and PyTorch*, O'Reilly Media, Inc., 2020. Available Online
- Kalnay, E. (2003). Atmospheric Modeling, Data Assimilation, and Predictability. Cambridge University Press. Available Online
- Ke, G., Meng, Q., & Yu, D. (2017). *Light: A highly efficient gradient boosting decision tree*. In *Advances in Neural Information Processing Systems* (NeurIPS), 30, Available Online
- Ke, G., Meng, Q., Finley, T., et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. Advances in Neural Information Processing Systems. Available Online
- Keras API documentation, "MobileNet, MobileNetV2, and MobileNetV3." Available Online
- M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A.

Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng, "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015. Available Online

- R. M. Felder and R. Brent, "Active Learning: An Introduction." Available Online
- Random forest as a generic framework for predictive modelling of spatial and spatio-temporal variables, Tomislav Hengl1, Madlene Nussbaum, Marvin N. Wright, Gerard B.M. Heuvelink4, Benedikt Gräler Available Online
- Ridge Regression for Manifold-valued Time-Series with Application to Meteorological Forecast, Esfandiar Nava-Yazdani Available Online
- Rudin, C. (2019). Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead. Nature Machine Intelligence, 1(5), 206-215. Available Online
- Rudin, C. (2019). *Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead*. *Nature Machine Intelligence*, 1(5), 206–215,Available Online
- Schultz, D. M., Doswell, C. A., & Bosart, L. F. (2021). *The meteorology of severe convective storms*. *Weather and Forecasting*, 36(6), 1671-1693, Available Online
- Wikipedia contributors, "Inceptionv3," *Wikipedia*, Feb. 2024. Available Online
- Wu, T., Huang, X., & Liu, J. (2020). *A deep learning-based approach for short-term wind speed prediction*. *Renewable Energy*, 147, 2993-3003, Available Online
- Yan, H., Li, S., & Zhang, L. (2018). *A deep learning model for rainfall prediction in urban areas*. *Journal of Hydrology*, 558, 1-12,Available Online
- Yang, S., Wang, Y., & He, J. (2017). *A review of machine learning for precipitation forecasting*. *Journal of Hydrology*, 547, 97-108,Available Online

- Yu, Y., Liu, Y., & Wang, Z. (2021). *A deep learning model for multi-step-ahead wind power forecasting*. *Renewable Energy*, 163, 362-373,Available Online

12. Appendices

11.1 Data Dictionary and Feature Descriptions

- Hour: Represents the time of day (0–23 hours). This feature captures diurnal variations in temperature, allowing models to recognize daily patterns.
- Day of Week: Encoded from 0 to 6, representing Monday to Sunday, this feature captures weekly variations in temperature, considering different patterns throughout the week.
- Latitude: Represents the geographical location of the station, playing a key role in distinguishing different climatic zones.
- Longitude: Similarly, this geographical feature helps provide spatial context to each temperature measurement, allowing for more accurate spatial modeling.
- Distance to Lake: The distance between each station and nearby lakes (in meters). This feature provides insights into how proximity to water bodies influences the temperature.
- Distance to River: Captures the distance between stations and nearby rivers, useful in modeling microclimate effects.
- Elevation: Elevation values (in meters) above sea level, providing information on how temperature changes with altitude.
- Temperature (Target Variable): The primary variable being predicted. It reflects the temperature reading at different geographic locations over time.
- Day of Year: Numeric feature representing the day (1–365), capturing seasonality effects across the dataset.
- Month: Encoded from 1 to 12, providing insight into seasonal variations.

11.2 Hyperparameter Settings

The models trained for this research required careful selection and tuning of hyperparameters. Here we provide an overview of the hyperparameter settings used for each of the key models.

11.2.1 Ridge and Lasso Regression

- Alpha: A regularization parameter that balances the strength of regularization. Multiple values (0.1, 1, 10) were tested to find the best fit.
- Solver: Both models utilized the 'auto' solver, which selects the best approach based on data size and complexity.

11.2.2 SVR (Support Vector Regressor)

- Kernel: Radial Basis Function (RBF) provided the best fit, capturing non-linear relationships in the data.
- C Parameter: The penalty parameter was optimized to balance error tolerance with regularization.
- Epsilon: Controlled the width of the margin where no penalty is given. A small epsilon (0.1) was used for higher precision.

11.2.3 LightGBM

- Number of Leaves: Set to 31, controlling the complexity of the learned model.
- Learning Rate: Initially set to 0.05 and tuned to 0.01 for further optimization to slow learning but achieve more accurate predictions.
- Max Depth: Limited to 7 to avoid overfitting and ensure efficient training given computational constraints.

11.2.4 CNN (Convolutional Neural Network)

- Number of Layers: 4 convolutional layers followed by 2 fully connected layers. Each convolutional layer applied ReLU activation.
- Filter Size: Set to 3x3 with stride of 1, aimed at capturing local spatial features.
- Pooling Layers: Max pooling applied with a 2x2 window to reduce spatial dimensions and computation.

11.2.5 LSTM (Long Short-Term Memory)

- Number of Units: 50 units per LSTM layer, allowing the model to capture temporal dependencies.
- Dropout Rate: 0.3 applied to reduce overfitting.
- Sequence Length: 7, allowing the model to capture one week of historical data for each prediction.

11.2.6 GCN (Graph Convolutional Network)

- Number of Layers: 2 graph convolutional layers with ReLU activation.
- Graph Structure: Based on station proximity, with a focus on spatial relationships.
- Learning Rate: 0.001 with Adam optimizer for smoother convergence.

11.2.7 Ensemble Models

- Weighted Average Weights: Determined empirically by comparing model performances. The ensemble weight for CNN and LSTM was set higher due to better performance over classical regression models.

11.3 Code and Usage Instructions

- **Repository**: All relevant code can be found in the **GitHub repository** Link The code includes:
    - Scripts for **data preprocessing**, **training models**, and **evaluating results**.
    - A **README** file explaining how to run experiments, including necessary installations and dependencies.
- **How to Run the Project**:
    - Install dependencies using the provided file:
    - Make sure all .shp , .shx files are in the working directory
    - Make sure all csv files are in the working directory
    - Run .ipynb files